

3.5.3 Repetition Statements: the for Loop and range()

لوپ (Loop) کوڈ کے ایک بلاک کو بار بار چلاتا ہے۔

CS XI

Chapter 3: Programming Fundamentals

Lecture 17 of 24

★ Golden

3.5.3 Repetition Statements: the for Loop and range()

★ Golden

LEARN IT

A **repetition statement (loop)** lets a block of code run **again and again**. This greatly reduces the need to write the same code many times. Each single run of the loop body is called an **iteration**.

1. The **for loop** is used when the **exact number of iterations is known**. It takes each value from a sequence one by one and stores it in the **loop variable** (e.g. `i`).

The `range()` function generates the sequence of numbers for the loop:

Syntax	How it works	Example	Values produced
<code>range(stop)</code>	From 0 up to, but not including , stop.	<code>range(3)</code>	0 1 2
<code>range(start, stop)</code>	From start up to, but not including, stop.	<code>range(2, 5)</code>	2 3 4
<code>range(start, stop, step)</code>	From start towards stop, jumping by step.	<code>range(1, 6, 2)</code>	1 3 5

```
for i in range(3):
    print(i)
```

OUTPUT

```
0
1
2
```

```
for i in range(1, 6, 2):
    print(i)
```

OUTPUT

```
1
3
5
```

```
for i in range(2, 5):
    print(i)
```

OUTPUT

```
2
3
4
```

```
for i in range(0, 6, 2):
    print("Even:", i)
```

OUTPUT

```
Even: 0
Even: 2
Even: 4
```

You can see all the values of a range at once by converting it to a list. A negative step counts **down**; if start equals stop, the range is empty:

```
print(list(range(5)))
print(list(range(2, 5)))
print(list(range(1, 6, 2)))
print(list(range(10, 0, -2)))
print(list(range(5, 5)))
```

OUTPUT

```
[0, 1, 2, 3, 4]
[2, 3, 4]
[1, 3, 5]
[10, 8, 6, 4, 2]
[]
```

WORKED EXAMPLE**Multiplication table of 7:**

```
for i in range(1, 6):
    print(f"7 x {i} = {7 * i}")
```

OUTPUT

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
```

Adding numbers with an accumulator. An **accumulator** is a variable that starts at 0 and collects a running total.

```
total = 0
for i in range(1, 11):
    total += i
print("Sum of 1 to 10 =", total)
```

OUTPUT

```
Sum of 1 to 10 = 55
```

i	1	2	3	4	5	...	10
total after	1	3	6	10	15	...	55

Looping over a string or list. A `for` loop can also take items directly from any sequence:

```
for ch in "Sindh":
    print(ch, end=" ")
print()
for m in [85, 90, 75]:
    print(m + 5)
```

OUTPUT

```
S i n d h
90
95
80
```

REMEMBER IT

range stops one short: `range(1, 11)` gives 1 to **10**. To include a number `n`, write `range(..., n + 1)`.
Number of values in `range(a, b)` = `b - a`.

COMMON MISTAKE

Writing `range(1, 10)` to print 1 to 10 misses 10 — an "off-by-one" logical error. Also, the loop body must be indented; a `for` line without an indented block gives `IndentationError: expected an indented block after 'for' statement`.

اردو میں سمجھیں

لوپ (Loop) کوڈ کے ایک بلاک کو بار بار چلاتا ہے۔ `for` لوپ تب استعمال ہوتا ہے جب پہلے سے معلوم ہو کہ کتنی بار دہرانا ہے۔ `range(stop)` صفر سے شروع ہوتا ہے، `range(start, stop)` سے، اور `range(start, stop, step)` ہر بار `step` کے برابر چھلانگ لگاتا ہے۔ آخری عدد (`stop`) کبھی شامل نہیں ہوتا۔

EXAM TIP

Expect "predict the output" questions on `range()` with a step, and programs such as "print a table", "sum of first n numbers", "even numbers from 1 to 20". Always state what range produces before tracing.

CHECK YOURSELF

1. Predict the output:

```
for i in range(10, 0, -3):
    print(i, end=" ")
```

2. Predict the output:

```
s = 0
for k in range(2, 11, 2):
    s += k
print(s)
```

3. Find the error:

```
for i in range(3):
    print(i)
```

Answers: 1. 10 7 4 1. 2. 30 (2 + 4 + 6 + 8 + 10). 3. The body is not indented: `IndentationError`. Indent `print(i)` by 4 spaces.

Next lecture: 3.5.3 The while Loop, break and continue